
ANALYSIS ON THE RELATIONSHIP BETWEEN RESOURCE USAGE AND TEMPERATURE ON RASPBERRY PI DEVICES

Louis Ryan Tan

30th Mar, 2025

created in  Curvenote

Link to Github page: <https://github.com/CuriousChum/jsc370>

Figures are located in the Appendix.

1 Introduction

1.1 Background

Edge computing devices such as Raspberry Pis have started gaining popularity due to their minimal design and affordable price tag. Unlike their bulkier counterpart, Raspberry Pis are small, yet powerful enough to act as a miniature personal computer (PC). This makes them suitable for specialized tasks such as gathering field data. However, due to their size and the rising demand in performance, the temperature within the device motherboard can rise quickly. The motherboard, being in the center of the device, is affected by the activity of peripheral devices and may wear out from intensive use.

This analysis aims to explore the relationship between device resource utilization and temperature across different Raspberry Pi specifications. The goal is to understand how different workloads affect device temperature (as measured by sensors in the motherboard) and whether this relationship can be used to predict thermal behavior under various application usages and resource utilization. This information is valuable for optimizing edge computing applications by anticipating device wear out and CPU throttling due to overheating.

1.2 Research Question

How does resource utilization affect device temperature across different Raspberry Pi configurations and application states, and can we identify patterns in this relationship to predict overall device temperature?

2 Methods

2.1 Dataset Description

The dataset used in this analysis was constructed and collected by research at the Queen's Telecommunication Research Lab (TRL), led by Ruslan Kain. It contains resource usage information from four heterogeneous Raspberry Pi 4 devices, with different RAM sizes (2GB, 4GB, 8GB) and CPU frequencies (1200MHz, 1500MHz, 1800MHz), measured under different application usages including gaming, streaming, augmented reality, mining, and idling. The data is collected in ~5 second intervals using the PsUtil Python package.

The dataset includes measurements from usages with random and periodic workloads using different Raspberry Pis. Each .tab file in the dataset contains data on a 'run' of a device. The tables are denoted as <device specification>_res_usage_data_*.tab. The runs included in this analysis are only from files that match the following pattern: <device specification>_res_usage_data_rvp_*.tab.

Table 1 lists all the variables regarding device resource information during each run of the Raspberry Pi 4s that is given in the dataset.

Table 1: Variable Descriptions

Variable Name	Description
<code>time_stamp</code>	Precise wall-clock time when the measurement was taken
<code>time</code>	Time as a floating-point number expressed in seconds since the Unix epoch (UTC)
<code>cpu_freq</code>	System-wide CPU cycle frequency (MHz)
<code>cpu</code>	System-wide CPU utilization as a percentage (%)
<code>cpu_user_time</code>	Time spent by normal processes executing in user mode, including guest time (seconds)
<code>cpu_idle_time</code>	Time spent doing nothing (seconds)
<code>cpu_system_time</code>	Time spent by processes executing in kernel mode (seconds)
<code>memory</code>	Memory currently in use or very recently used, as a percentage of RAM (%)
<code>net_sent</code>	Total bytes sent since boot
<code>net_recv</code>	Total bytes received since boot
<code>net_upload_rate</code>	Bytes sent in the last interval, divided by the interval length (MB/s)
<code>net_download_rate</code>	Bytes received in the last interval, divided by the interval length (MB/s)
<code>temp</code>	Device temperature (°C)
<code>wifi_freq</code>	Operating frequency / channel of the Wi-Fi interface (GHz)
<code>bit_rate</code>	Current Wi-Fi bit rate (bit/s)
<code>link_quality</code>	Overall Wi-Fi link-quality metric (contention, interference, error rate, etc.)
<code>link_quality_max</code>	Maximum possible value of the Wi-Fi link-quality metric
<code>gpu</code>	GPU utilization as a percentage (%)
<code>gpu_memory</code>	GPU memory in use or very recently used (%)
<code>state</code>	Resource-usage state of the device associated with the running application (e.g., <i>mining</i> , <i>streaming</i> , <i>idle</i>)

2.2 Data Wrangling

The dataset contains a lot of columns, some of which are redundant, filled with only zeros, or describes irrelevant information; these columns will thus be dropped from the analysis.

First, columns describing CPU time spent will be dropped since the focus is mainly on overall CPU utilization, so `cpu` and `cpu_freq` is sufficient. Manual inspection of the data shows that the columns `gpu` and `gpu_memory` are always zero, implying that the builtin GPU is not utilized in any of the workloads, so it is drop. Furthermore, the columns `link_quality_max` and `link_quality` are dropped because of the lack of documentation; the exact descriptions cannot be found in the `PsUtil` package. Finally, the columns `wifi_freq`, `net_sent`, and `net_recv` are dropped because these are already better represented by `net_upload_rate` and `net_download_rate`; note that `wifi_freq` is not indicative of device resource usage and both `net_*` features are cumulative sums of the `net_*_rates`, respectively.

Next, the tables from the dataset will be combined while preserving per-device and per-run information using the columns `pi_id` and `run_id`, respectively. For brevity, the devices will be denoted as follows

- `dev0`: Raspberry Pi 4, 2GB RAM, 1200 MHz
- `dev1`: Raspberry Pi 4, 2GB RAM, 1500 MHz

Table 2: Pruned Variable Descriptions

Variable Name	Description
cpu	System-wide CPU utilization as a percentage (%)
cpu_freq	System-wide CPU cycle frequency (MHz)
state	Label feature for the resource-usage state of the device associated with the running application
memory	Memory currently in use or very recently used, as a percentage of RAM (%)
net_upload_rate	Number of bytes sent in the last time interval, divided by the time interval (MBytes/s)
net_download_rate	Number of bytes received in the last time interval, divided by the time interval (MBytes/s)
temp	Device temperature (°C)
pi_id	Device id, one of dev0, dev1, dev2, dev3
run_id	Run id, ranges from 000 to 018

- dev2: Raspberry Pi 4, 4GB RAM, 1500 MHz
- dev3: Raspberry Pi 4, 8GB RAM, 1800 MHz

Meanwhile, `run_id` ranges from 000 to 018, each table in the initial dataset is considered a different run, irrespective of the device.

The final pruned dataset contains columns that is described in Table 2.

2.3 Exploratory Data Analysis

Since each run is a time-series data, comparing each feature directly might not be ideal. Instead, the comparison will either be done over time, or on the differences.

2.3.1 Comparison over Time

Since it's infeasible to visualize and compare all runs with more than 30,000 observations, we instead sample one run from different devices and look at random sections within it. Figure 1, Figure 2, Figure 3, Figure 4 are sections of runs that are chosen uniformly at random for each device.

The plots shows several notable things: device temperature tends to move to a certain semi-constant value on each state but with a noticable jitter. Similarly, CPU usage tends to move to a constant value with noticable jitters and occasional spikes every state. CPU frequency, on the other hand, appears to erratically jump from high to low, this is a phenomenon called throttling, where the CPU dynamically changes its frequency due to some circumstance, either due to low usage (as to preserve energy), or due to overheating, which is not seen here. Finally, we observe that there is only network activity when the device is streaming. Finally, memory usage is consistent within states with a more noticable variation when the device is streaming or running an augmented reality application.

2.3.2 Feature Distributions

Then, we visualize the distribution of features per device colored by state. The following figures shows the aggregated distribution over all runs. Network upload/download rates are scaled up to the scale of KBps in this visualization due to be more noticable.

There are notable patterns in the distribution of each state: temperature and CPU usage is approximately unimodal for each state; CPU frequency in each state are either maximum or a specific value with not much in between; memory usage being quite consistent with similar centers, albeit having different spread/variance between device; network is activity is dominantly zero.

2.3.3 Intra-Feature Relationship

Looking at several relationships between features, we see consistent relationship as seen before. Most notably, the clusters when plotting CPU usage vs. temperature as seen below in Figure 9.

2.3.4 Feature Engineering

Motivated by findings in the EDA phase, we will create the following columns as a means to transform the original features:

- `cpu_freq_mean`: the average CPU frequency within a period of a state in each run per device. Since the CPU throttles to save power on low intensity tasks, it will produce less heat as a consequence; however, depending on how long it runs on the higher or lower frequency, the thermal contribution may vary.
- `cpu_ma`: the moving average (MA) of CPU utilization. The CPU measurements might contain a lot of noise due since the method of measuring CPU utilization is usually sampling. Taking the moving average helps reduce noise in our data. The window chosen for this is 4 lags. The reasoning is that too much lag will exacerbate the effect of jumps in CPU activity, oversmoothing it.
- `temp_ma`: the moving average of device temperature. Similar to CPU usage, temperature measurements might not be very accurate. The window chosen is 4 lags as well, with the same reasoning to prevent oversmoothing.
- `temp_ma_lag`: a one interval (5s) lag from `temp_ma`. Used as a predictor since temperature is naturally autoregressive; the current temperature will be similar to the temperature sometimes close in the past.
- `temp_ma_diff`: the differences in `temp_ma`. This is used as another possible response when modelling due to the high correlation between `temp_ma` and `temp_ma_lag`.

Network activity metrics used will be the one in KBps, since their value are extremely small.

Figure 10 and Figure 11 are some visualizations of the newly created features.

As seen from the figures above, the moving averages are smoother and less noisy, also `temp_ma_diff` being centered and symmetric, as expected since the temperature data tends to stabilize.

2.4 Modelling

In this phase, we will attempt to build a model that can predict device temperature under specific workloads. Due to our data being highly structured, we can expect to have accurate models at the expense of generalization; our models will be good at predicting temperature under similar workloads, but might be very inaccurate for workloads that are dissimilar to these usages.

Furthermore, the model will be trained to predict the temperature moving average, this is to reduce the noise that is learned by the model that might lead to overfitting.

The feature state heavily impacts resource utilization, as seen it the exploration phase, but it constrains the workloads that the model can analyze and potentially reduces generalization. Hence, it will be removed from the final model; but another model purely based on `temp_ma_lag`, `pi_id`, and `state` is used as a baseline comparison model, but not used in any model that is considered.

To begin, we perform some analysis to reduce the number of features in hopes of improving model generalizations. The first feature we examine is `pi_id`. We wish to check whether there are meaningful differences between the different configurations in CPU frequency and RAM size. To do this, we perform a likelihood ratio test between a linear mixed model that uses `pi_id` and one that doesn't.

The likelihood ratio test indicates that indeed there is a difference between devices, so this feature will be kept. The details of the likelihood ratio tests are shown in Table 3.

Table 3: Likelihoods and LRT Statistics

LR Statistics	χ^2 Degree of Freedom	p value
128.92	3	<1e-16

Table 4: resource model feature importances when predicting temperature MA

Feature Name	Value
memory	0.000110
pi_id_dev0	0.000000
pi_id_dev1	0.000000
pi_id_dev2	0.000000
pi_id_dev3	0.000000
net_upload_rate_kbps	0.000072
net_download_rate_kbps	0.000052
cpu_ma	0.089091
cpu_freq_mean	0.011228
temp_ma_lag	0.899446

We train an XGBoost regression model on a run-aware, time-ordered split of the data. Each contiguous recording (run_id) is treated as an independent block: earlier parts of the run forms the training set and the following part becomes the validation set in a walk-forward loop. Furthermore, we perform cross validation using a time series split with 6 splits and using an hour interval as validation set size.

The following tables shows the feature importances for the resource model and the state model when predicting temp_ma. The average MSE over all cross validation steps is 0.177 and 0.13 for the resource model and the state model respectively. However, both models show that temp_ma_lag is the dominant feature while all other features have little importance scores. This might be because stark temperature changes happen less often compared to slow temperature changes, which means the lagged moving average will be a very good predictor. For example, if the temperature have stabilized because the state hasn't changed for a while, the lagged temperature is approximately equal to the current temperature.

To attempt to account for the strong correlation between the lag and the current temperature moving average and extract more predictive power for resource activity data we train two more models with both setups again, but predicting temp_ma_diff instead.

The following are the feature importances for the new models. The average MSE across cross validation runs are 0.0434 and 0.0488, respectively. The lower MSE is due to the differences in temperature MA are small.

Table 5: state model feature importances when predicting temperature MA

Feature Name	Value
temp_ma_lag	0.915936
state_augmented_reality	0.000320
state_game	0.004800
state_idle	0.008379
state_mining	0.070512
state_stream	0.000051
pi_id_dev0	0.000000
pi_id_dev1	0.000000
pi_id_dev2	0.000000
pi_id_dev3	0.000000

Table 6: resource model feature importances when predicting temperature MA difference

Feature Name	Value
memory	0.137160
pi_id_dev0	0.000000
pi_id_dev1	0.000000
pi_id_dev2	0.000000
pi_id_dev3	0.000000
net_upload_rate_kbps	0.064952
net_download_rate_kbps	0.022611
cpu_ma	0.184090
cpu_freq_mean	0.226223
temp_ma_lag	0.364965

Table 7: state model feature importances when predicting temperature MA difference

Feature Name	Value
temp_ma_lag	0.331357
state_augmented_reality	0.203710
state_game	0.110747
state_idle	0.084816
state_mining	0.244647
state_stream	0.024724
pi_id_dev0	0.000000
pi_id_dev1	0.000000
pi_id_dev2	0.000000
pi_id_dev3	0.000000

It is interesting to note, however, all XGBoost models doesn't seem to think that differences in device is important for prediction. Meanwhile the likelihood ratio test shows a different result. A hypothesis is that the differences in devices are sufficiently modelled by other features such as CPU frequency and memory usage.

3 Summary

We see that there is stark difference in CPU and memory usage for each state with each state utilizing around the same percentage of CPU utilization for each of their usage window. However, the utilization percentage for each state is similar across devices. We also observe CPU throttling in `cpu_freq`, where the CPU scales down its frequency to conserve power usage when running processes with lower utilization.

Furthermore, the temperature seems to increase/decrease quickly when the CPU utilization changes rapidly due to change of device application state, and plateauing to a stable temperature that is consistent for each state. This might imply that the device temperature change has some relationship to CPU utilization.

4 Appendix

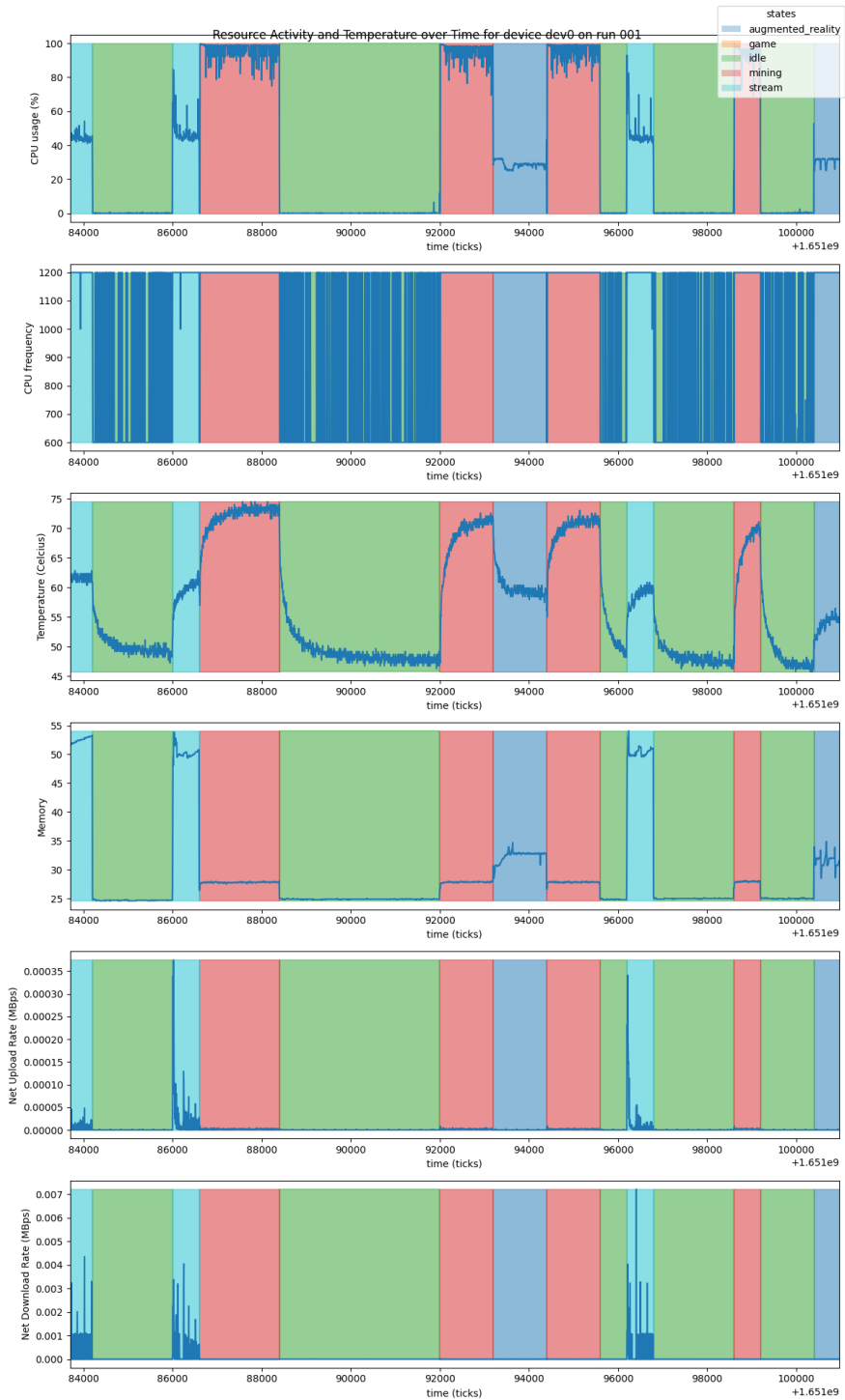


Figure 1: plots of several resource activity samples for dev0

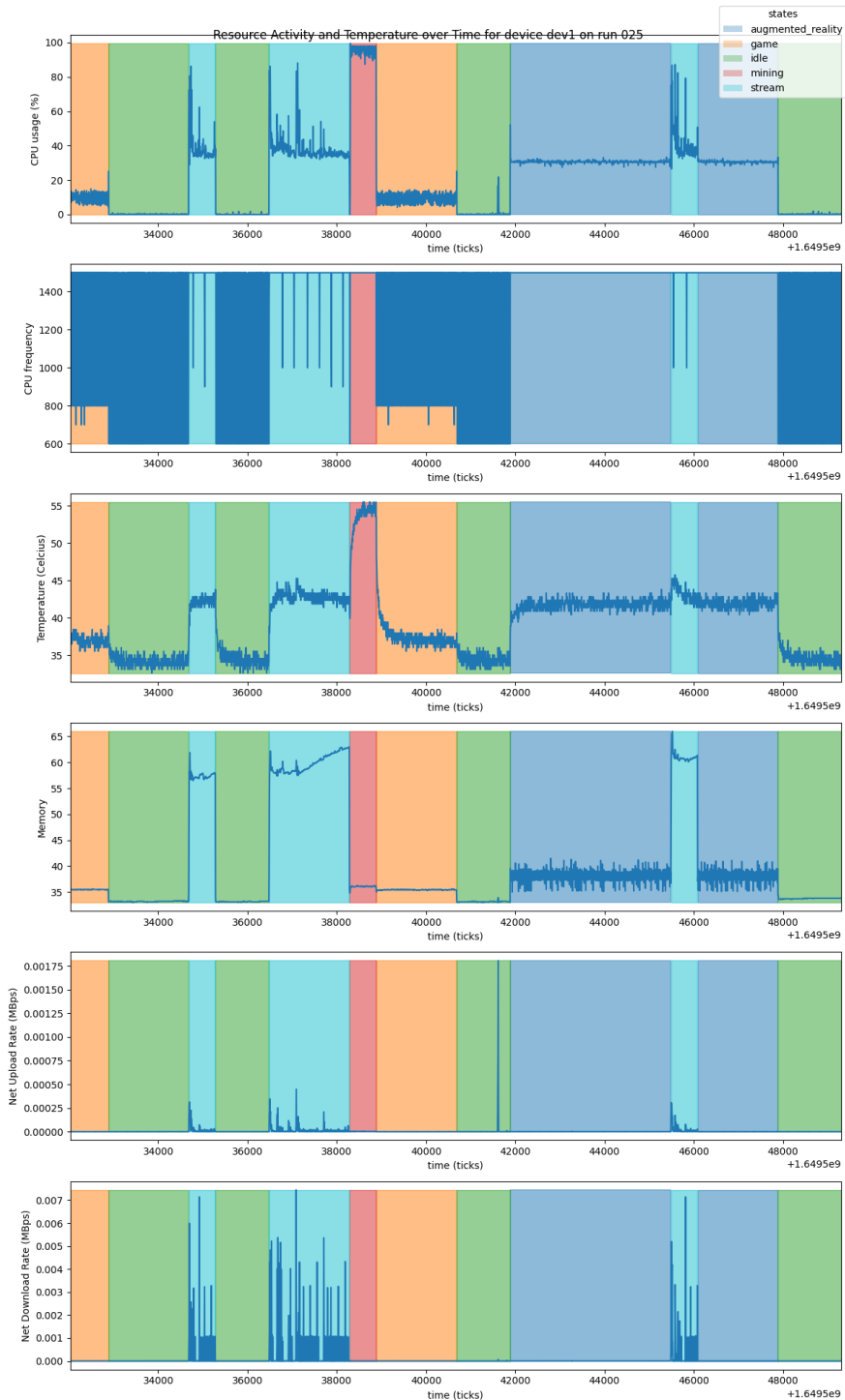


Figure 2: plots of several resource activity samples for dev1

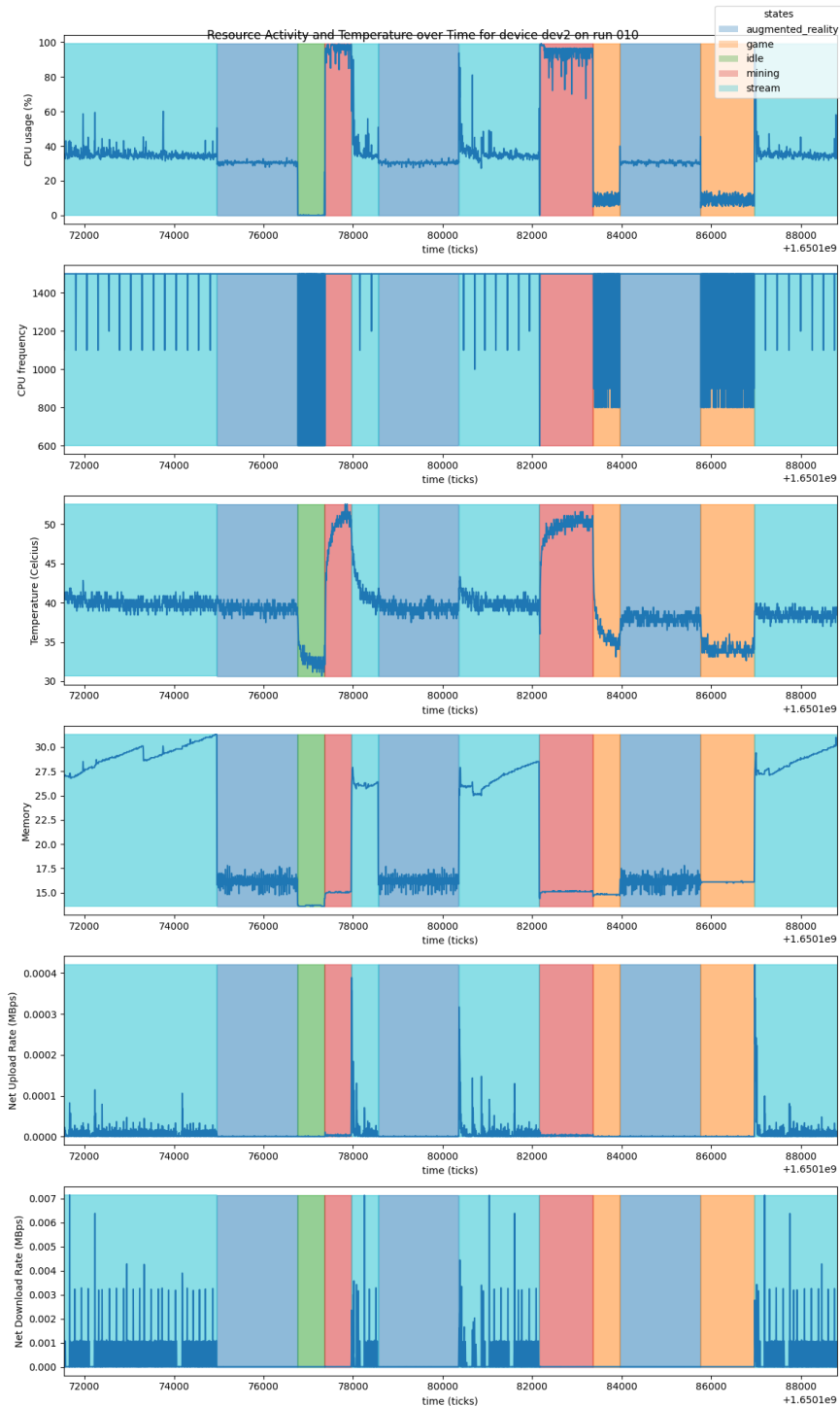


Figure 3: plots of several resource activity samples for dev2

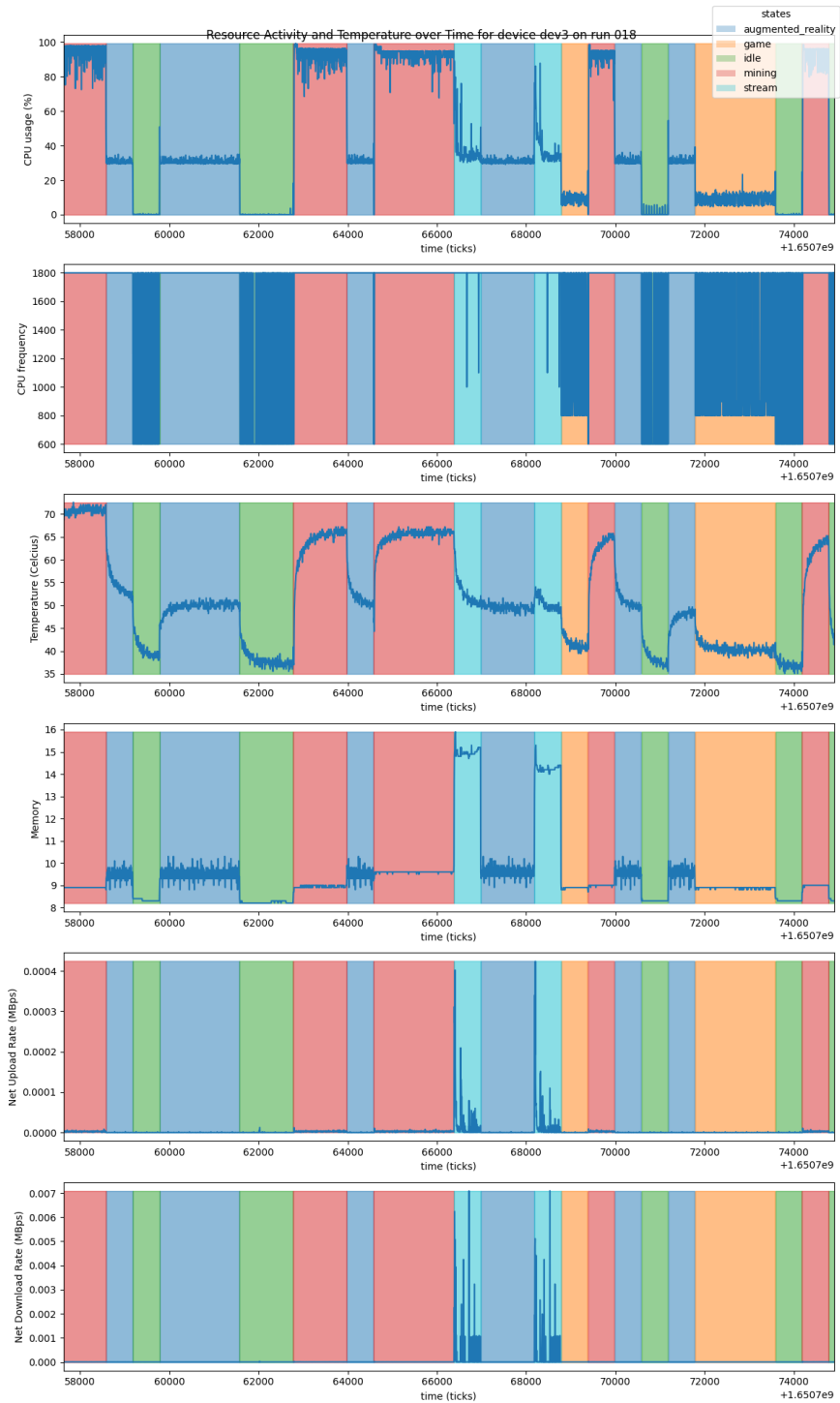


Figure 4: plots of several resource activity samples for dev3

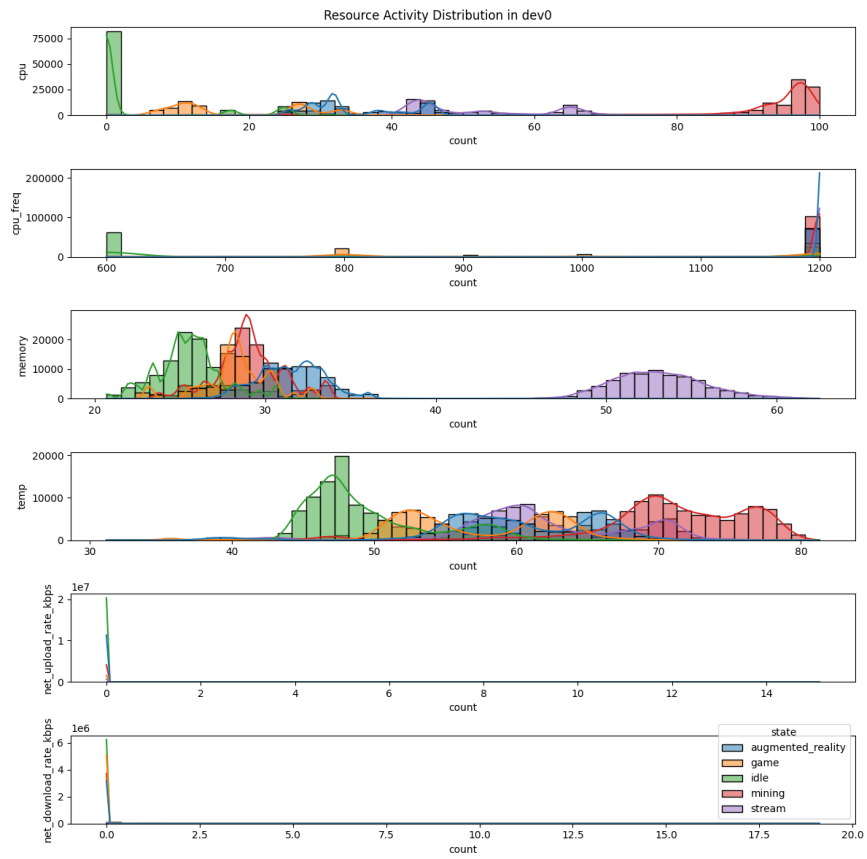


Figure 5: resource activity distribution for dev0

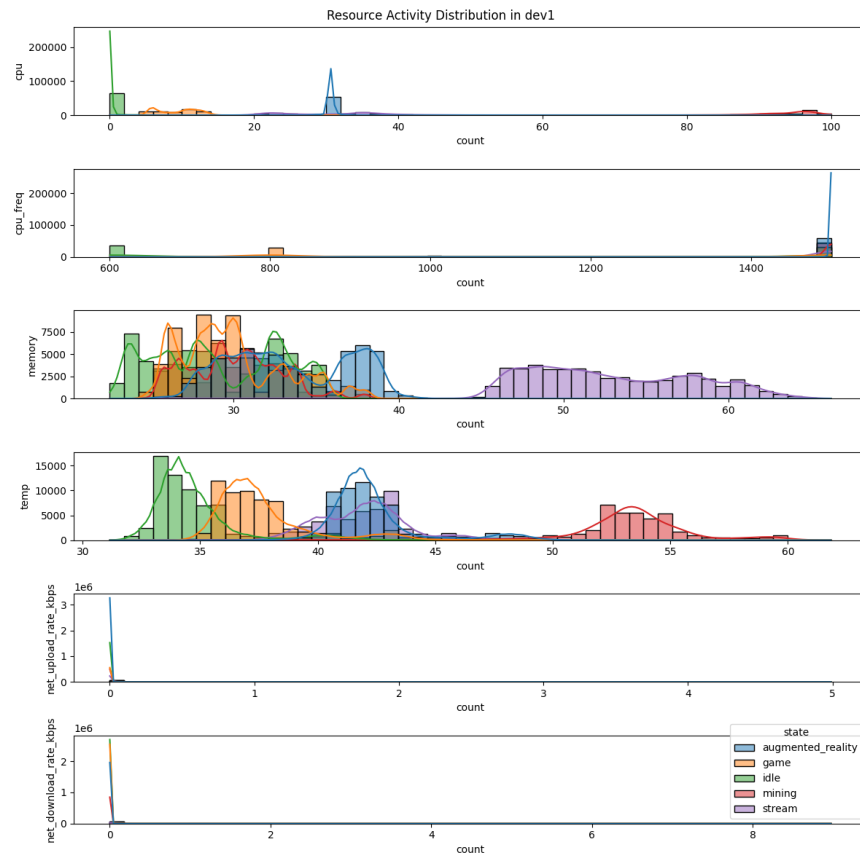


Figure 6: resource activity distribution for dev1

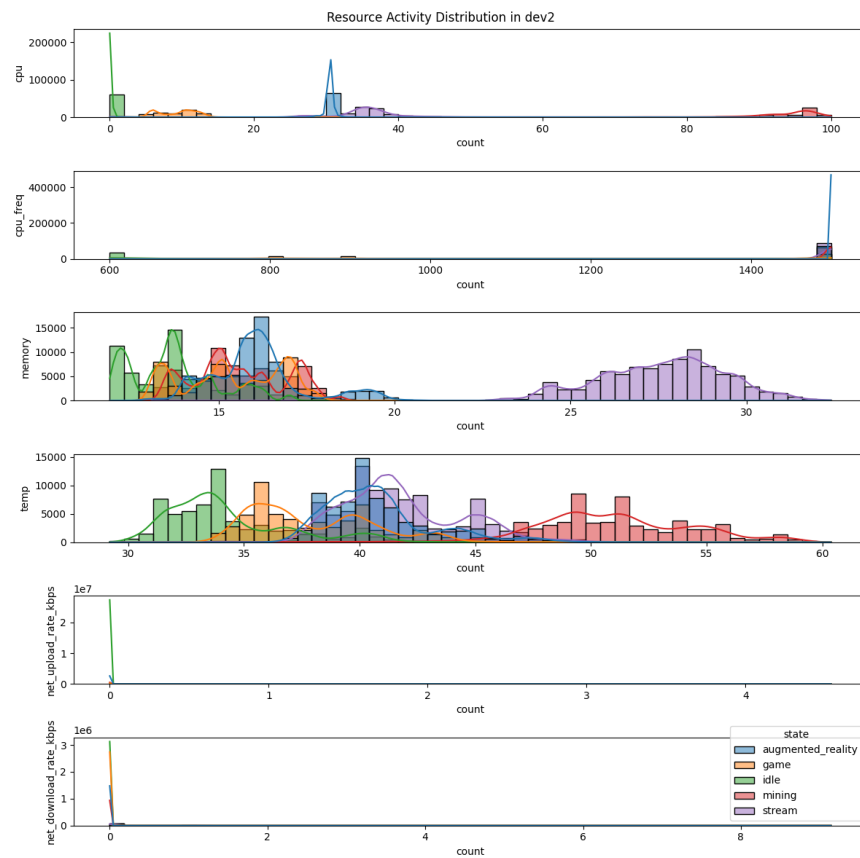


Figure 7: resource activity distribution for dev2

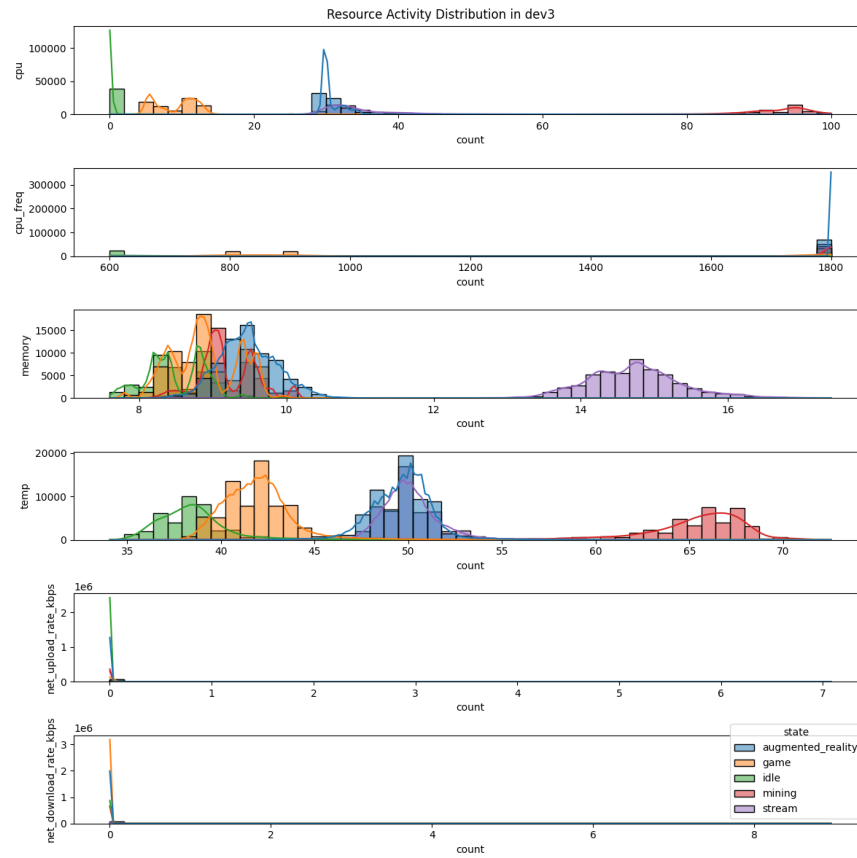


Figure 8: resource activity distribution for dev3

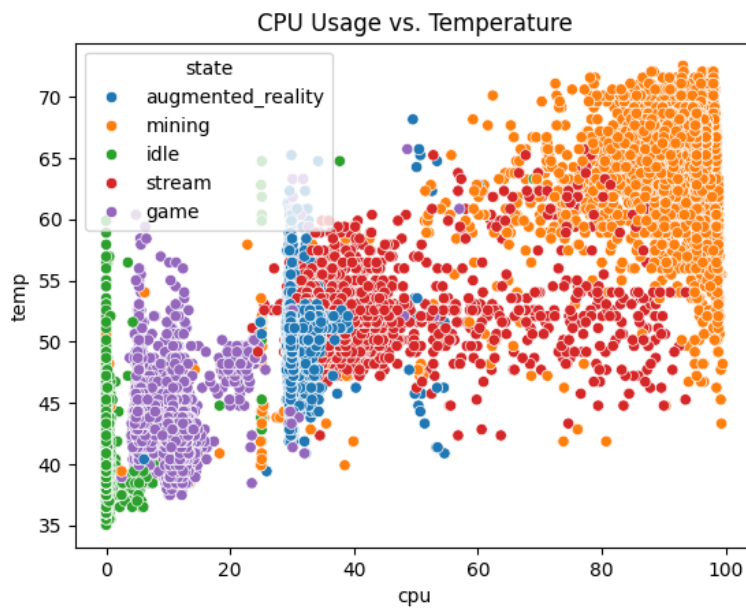


Figure 9: CPU usage vs. Temperature on dev3

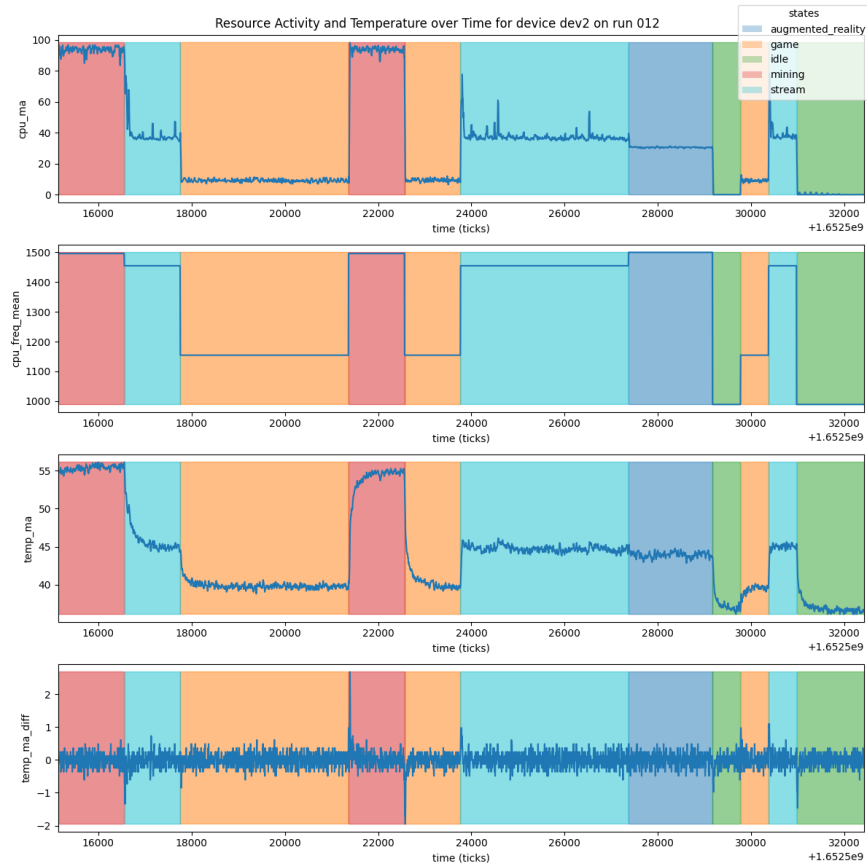


Figure 10: plot of samples of engineered features on dev2

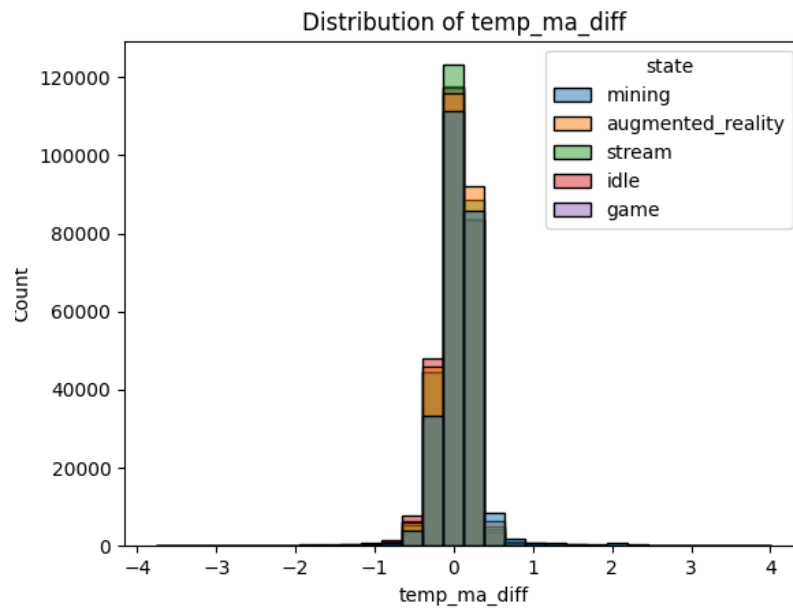


Figure 11: distribution of temp_ma_diff